

Network Routing

Rakesh Itani
Master's of Science in Computer Science
Idaho State University
Pocatellom ID, 83209 USA
rakeshitani@isu.edu

March 2026

Abstract

This project report summarizes the use of Dijkstra's algorithm to find paths through a graph representing a network routing problem.

1 Introduction

Dijkstra's algorithm is widely used algorithm in graph theory for finding the shortest path between nodes in a graph with non-negative edge weights.

Dijkstra's algorithm relies heavily on the use of priority queue to efficiently select the next node to visit during exploration of the graph. In this project, two of such priority queues will be used: array and heap, to analyze the Asymptotic notation of Dijkstra's algorithm.

2 Pseudocode and Asymptotic Analysis

2.1 Pseudocode for unsorted array priority queue

```
1  def dijkstra_implementataion(self, srcIndex,
    use_heap=False):
2      pq = object of array queue if not use_heap else
        object of binary heap queue
3      # initializing the prev array to nil
4      self.prev_array = [None] *
        len(self.network.nodes)
5
6      # Start with the source node and updating the
        dist value to 0
7      pq.insert(srcIndex)
8      pq.decrease_key(0, srcIndex)
9
10     # iterate until there is node in queue
11     while pq.queue:
12         curr_min_node_index = pq.delete_min()
13         curr_node =
            self.network.nodes[curr_min_node_index]
14
15         # Check each edge from the lowest distance
            node
16         for edge in curr_node.neighbors:
17             node_index = edge.dest.node_id
18             new_distance =
                pq.dist[curr_min_node_index] +
                edge.length
19             if new_distance < pq.dist[node_index]:
20                 pq.insert(node_index)
21                 self.prev_array[node_index] =
                    curr_min_node_index
22                 pq.decrease_key(new_distance,
                    node_index)
```

Time complexity and space complexity

Using unsorted Array

1. insert method takes $O(1)$ time and $O(1)$ space; this method appends new node to the queue
2. decrease_key method takes $O(1)$ time and $O(1)$ space; this method updates the distance of node
3. delete_min method takes $O(n)$ times and $O(1)$ space; this method scans the queue and return the the node with minimum distance value

$$Time_complexity = O(1) + O(1) + O(n) * (O(1) + O(n)) \approx O(n^2) \approx O(|V|^2)$$

$$Space_complexity = O(n) \approx O(|V|)$$

where, V is verticies in graph

(1)

Using binary heap

1. insert method takes $O(\log n)$ time and $O(1)$ space; this method appends new node to the queue
2. decrease_key method has two function calls
bubble_up takes $O(\log n)$ times; this method switch the parent and child node in a tree of depth $O(\log n)$
swap_nodes takes $O(1)$ time
and space complexity is $O(1)$ space
3. delete_min method has two function calls
bubble_down takes $O(\log n)$ times; this method switch the parent and child node in a tree of depth $O(\log n)$
swap_nodes takes $O(1)$ time
space complexity is $O(1)$ space

$$Time_complexity = O(1) + O(\log n) + O(n) * (O(\log n) + O(\log n)) \approx O(n \log n) \approx O(|V| \log |V|)$$

$$Space_complexity = O(n) \approx O(|V|)$$

where, V is verticies in graph

(2)

2.2 Pseudocode for unsorted array priority queue

```
1  class ArrayPriorityQueue:
2      def __init__(self, nodes):
3          # initializing the dist array to infinite
4          self.dist = [math.inf] * len(nodes)
5          self.queue = []
6
7          # adds new item in queue
8      def insert(self, node_index):
9          self.queue.append(node_index)
10
11         # updates the distance value of a node
12     def decrease_key(self, dist, node_index):
13         self.dist[node_index] = dist
14
15         # finds a node with minimum distance and return
16         # it after removing it from the queue
17     def delete_min(self):
18         if not self.queue:
19             return None
20
21         min_node = None
22         min_distance = math.inf
23
24         for node in self.queue:
25             if self.dist[node] < min_distance:
26                 min_node = node
27                 min_distance = self.dist[node]
28
29         self.queue.remove(min_node)
30         return min_node
```

Time complexity and space complexity

1. initialization takes $O(1)$ times to initialize the dist, and queue, and $O(n)$ space to store the dist, and queue
2. insert method takes $O(1)$ time and $O(1)$ space
3. decrease_key method takes $O(1)$ time and $O(1)$ space
4. delete_min method takes $O(n)$ times and $O(1)$ space

$$Time_complexity = O(1) + O(1) + O(1) + O(n) \approx O(n) \approx O(|V|)$$

$$Space_complexity = O(n) + O(1) + O(1) + O(1) \approx O(n) \approx O(|V|) \quad (3)$$

where, V is vertices in graph

2.3 Pseudocode for binary heap priority queue

```
1  class HeapPriorityQueue:
2  def __init__(self, nodes):
3      # initializing the distance to infinity
4      self.dist = [math.inf] * len(nodes)
5      self.queue = []
6      # initializing the pointer of the tree to
7      # infinity
8      self.q_pointer = [math.inf] * len(nodes)
9
10     # add a node into the heap
11     def insert(self, node_index):
12         self.queue.append(node_index)
13         self.q_pointer[node_index] = len(self.queue) - 1
14         self.bubble_up(len(self.queue) - 1)
15
16     # adjust the distance update of nodes
17     def decrease_key(self, dist, node_index):
18         self.dist[node_index] = dist
19         self.bubble_up(self.q_pointer[node_index])
20
21     # bubble_up maintains the property of heap after
22     # insertion or decrease key operation
23     def bubble_up(self, child_index):
24         while child_index > 0:
25             parent_index = (child_index - 1) // 2
26             if (
27                 self.dist[self.queue[child_index]]
28                 >= self.dist[self.queue[parent_index]]
29             ):
30                 break
```

```

29         self.swap_nodes(child_index, parent_index)
30         child_index = parent_index
31
32     # return the minimum element from the heap and
    removes it
33 def delete_min(self):
34     if not self.queue:
35         return None
36
37     min_index = self.queue[0]
38     if len(self.queue) == 1:
39         self.queue.pop()
40         return min_index
41
42     last_index = len(self.queue) - 1
43     # swaps the item at the end with root and return
    the root node
44     self.swap_nodes(0, last_index)
45     self.queue.pop()
46     self.bubble_down(0)
47
48     return min_index
49
50 # restores the heap property starting from the given
    parent_index
51 def bubble_down(self, parent_index):
52     while True:
53         left_child_index = 2 * parent_index + 1
54         if left_child_index >= len(self.queue):
55             break
56
57         child_index_left = left_child_index
58         child_index_right = left_child_index + 1
59         if (
60             child_index_right < len(self.queue)
61             and self.dist[self.queue[child_index_right]]
62             < self.dist[self.queue[left_child_index]]
63         ):
64             child_index_left = child_index_right
65
66     # checking the distance of parent node and the
    smallest child node
67     if (
68         self.dist[self.queue[parent_index]]
69         <= self.dist[self.queue[child_index_left]]
70     ):

```

```
71         break
72
73     self.swap_nodes(parent_index, child_index_left)
74     parent_index = child_index_left
75
76     # perform the swap between nodes
77     def swap_nodes(self, i, j):
78         self.queue[i], self.queue[j] = self.queue[j],
79         self.queue[i]
80         self.q_pointer[self.queue[i]] = i
81         self.q_pointer[self.queue[j]] = j
```

Time Complexity and space compelxity

1. initialization takes $O(1)$ times to create the dist, queue, and pointer, and $O(n)$ space to store the dist, queue, and pointer
2. insert method takes $O(\log n)$ time and $O(1)$ space
3. decrease_key method has two function calls
bubble_up takes $O(\log n)$ times
swap_nodes takes $O(1)$ time
and space complexity is $O(1)$ space
4. delete_min method has two function calls
bubble_down takes $O(\log n)$ times
swap_nodes takes $O(1)$ time
space complexity is $O(1)$ space

$$Time_complexity = O(1) + O(\log n) + O(\log n) + O(\log n) \approx O(\log n) \approx O(\log |V|)$$

$$Space_complexity = O(n) + O(1) + O(1) + O(1) + O(1) \approx O(n) \approx O(|V|)$$

where, V is vertices in graph

(4)

3 Observations and Results

3.1 Observation Table

Queue Type	Size	1	2	3	4	5	Mean time
Array	100	0.0003	0.0006	0.0004	0.0003	0.0003	0.0004
Array	1000	0.0109	0.0096	0.0119	0.0095	0.0101	0.0104
Array	10000	1.4635	1.4386	1.4543	1.4474	1.4821	1.4572
Array	100000	135.5281	122.2875	141.2688	165.9068	130.2860	139.0554
Binary Heap	100	0.0006	0.0011	0.0008	0.0006	0.0005	0.0007
Binary Heap	1000	0.0050	0.0050	0.0045	0.0055	0.0051	0.0050
Binary Heap	10000	0.2163	0.2152	0.2168	0.2160	0.2180	0.2165
Binary Heap	100000	1.1646	1.0881	1.3380	1.1206	1.1153	1.1652
Binary Heap	1000000	21.2837	20.0075	20.0632	20.4115	20.2848	20.4101

Table 1: Time taken to run Dijkstra’s Algorithm in different priority queue

The results shown in Table 1 present the running time of Dijkstra’s algorithm using different priority queue implementations for various graph sizes. Each experiment was performed using different random seeds while keeping the source and target nodes fixed.

From the data we can observe that the running time for the array-based priority queue increases rapidly as the graph size grows. In particular, when the size increases from 10,000 to 100,000 nodes, the running time increases by roughly two orders of magnitude. This trend suggests that the array implementation grows approximately quadratically with respect to the number of vertices.

Using this observation, we can estimate the expected running time for an array implementation when the graph size is 1,000,000 by extrapolating from the average running time measured for size 100,000 using the following formula.

$$\begin{aligned}
 \text{Array queue of size } 1000000 &= ((1000000/100000)^2) * \text{avg time for array of size } 100000 \\
 &= 100 * 139.0554 \\
 &= 13905.544 \text{ sec}
 \end{aligned}$$

The estimated running time of 13,905.544 seconds (approximately 3.86 hours) indicates that the array-based implementation becomes highly inefficient as the graph size increases.

The binary heap implementation is faster because selecting the minimum-distance vertex requires $O(\log V)$ time instead of $O(V)$ time in the unsorted array implementation, significantly improving the overall performance of Dijkstra’s algorithm for large graphs.

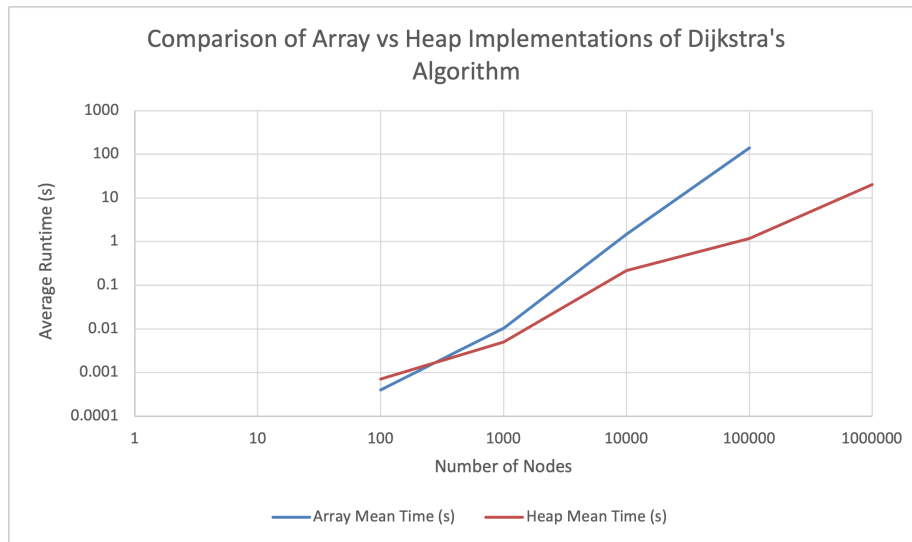


Figure 1: Plot between the different sample sizes and the mean time taken by Heap vs Array

Figure 1 shows the time taken by Dijkstra's algorithm using the array priority queue vs the Dijkstra's algorithm using the binary heap. We can observe that the binary heap can compute the Dijkstra algorithm faster than the array queue. As the size of sample increases, the binary heap computes the tree comparatively faster i.e. takes less time.

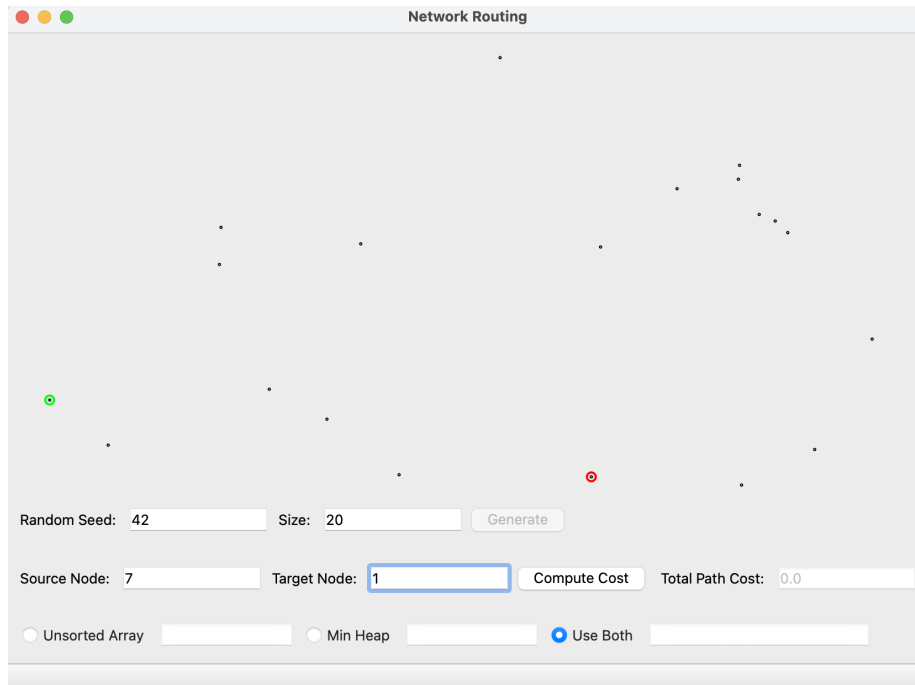


Figure 2: Graph for seed 42, size 20, source node : 7 and target node: 1

The figure (in figure 2) shows the 20 sample generated using seed 42. The source node is 7 and the target node is 1. But the target was unreachable from the given source. Thus, I could not include the time taken by the program and conclude that the target is unreachable.

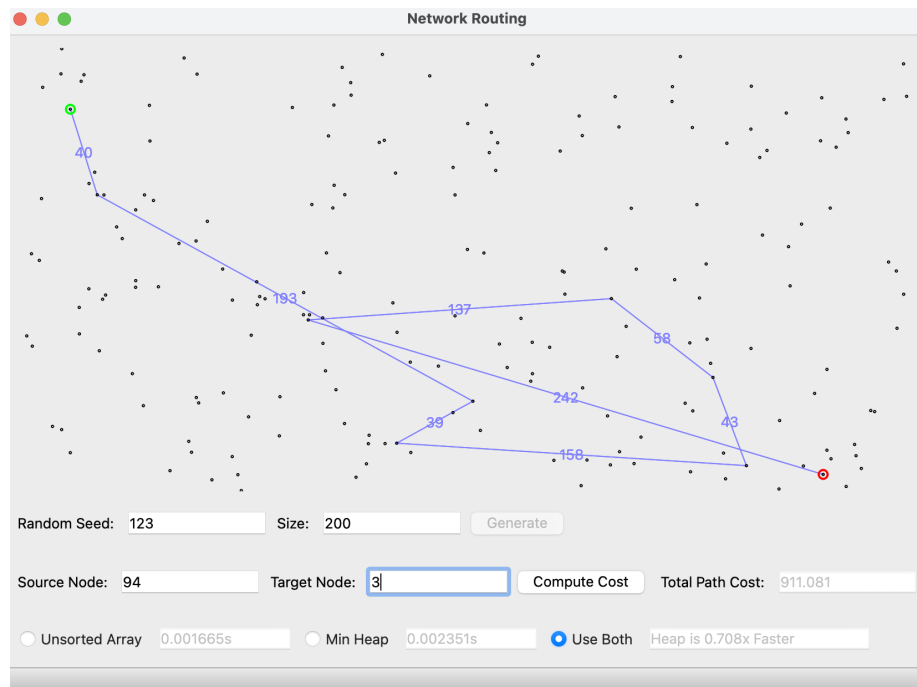


Figure 3: Graph for seed 123, size : 200, source node: 94 and target node: 3

The figure (in figure 3) shows the 200 sample generated using seed 123. The source node is 94 and the target node is 3. The total time taken to compute the Dijkstra's algorithm using array is 0.00167 sec and for binary heap it took 0.00235 sec i.e. the heap is 0.708 times faster. And the total cost of tree is 911.081.

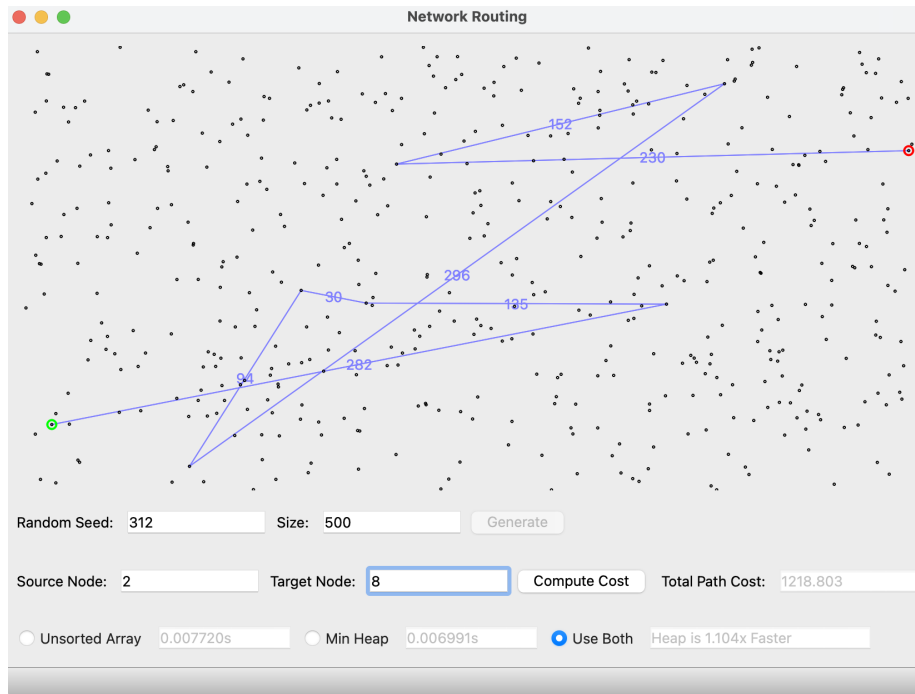


Figure 4: Grpah for Random Seed 312, Size : 500, Source Node: 2 and Target Node: 8

The figure (in figure 4) shows the 500 sample generated using seed 312. The source node is 2 and the target node is 8. The total time taken to compute the Dijkstra's algorithm using array is 0.00772 seconds and for binary heap it took 0.00699 seconds i.e. the heap is 1.104 times faster. And the total cost of tree is 1218.803.

4 Conclusion

This project was successfully able to generate the code to compute the Dijkstra's algorithm using the array and binary priority queue. This project was also able to estimate the time taken to compute the sample size of 1000000 for array queue. Also, the time complexity of both the priority queues have been successfully studied and analyzed.